Technical Design Document — FeeFlow

- GitHub Repository: https://github.com/Vedant1703/feeflow (Public)
- Live Deployment URL: https://feeflow.vercel.app
- Candidate: Vedant Vinod Kulkarni
- Problem Statement: Brief 4 — Coaching Institute's Fee Tracking System and Parent Portal (Company A, Jaipur)
- Evaluation Baseline: Node.js v20+, Next.js 14 App Router, TypeScript, Prisma ORM, PostgreSQL

1. Executive Summary & Problem Context

Company A operates four coaching centres across Jaipur (Vaishali Nagar, Malviya Nagar, Tonk Road, and Ajmer Road) serving over 1,200 students preparing for JEE and NEET competitive entrance examinations.

Prior to FeeFlow, the institute faced significant operational challenges:

- High volume of parent queries regarding instalment due dates, ledger balances, and payment receipts.
- Unmatched bank transfers and UPI payments sitting in unallocated suspension accounts.
- Inconsistent and disputed fee refund calculations upon student withdrawal.
- Aggressive or mistimed fee collection reminders that caused parent friction and damaged trust.
- Late identification of at-risk students likely to drop out before completing their courses.

FeeFlow was designed and engineered as a mission-critical financial operating system and mobile-first parent portal. It enforces strict double-entry ledger immutability, zero-client payment trust, MoE 2024 statutory refund compliance, evaluate-at-send bilingual reminders, and explainable academic risk heuristics.

2. Architecture & Rationale

2.1 Modular Monolith with Hexagonal Boundaries

- Next.js App Router Monolith: A unified TypeScript codebase hosting parent mobile experiences (`/parent/*`), staff administrative workspaces (`/staff/*`), webhook receivers (`/api/webhooks/*`), and background processing workers. This eliminates distributed transaction hazards, serialization overhead, and microservice network latency while ensuring end-to-end type safety between database schemas and client interfaces.
- Strict Hexagonal Isolation: Domain logic (`src/domain/`) contains zero external I/O, database drivers, or framework dependencies. All financial arithmetic, ledger rebase calculations, pro-rata refund formulas, candidate matching algorithms, reminder eligibility rules, and risk heuristics are implemented as deterministic, pure functions.
- Typed Infrastructure Ports: External dependencies (Razorpay Payment Gateway, MSG91 OTP/SMS Gateway, OAuth Identity Providers, Clock, PDF Generators) are isolated behind strict TypeScript interfaces with deterministic test adapters.

2.2 Security, Cryptographic Authentication & RBAC Layer

- Cryptographic OTP Service: Strict Indian E.164 normalization (`+91XXXXXXXXXX`), 6-digit CSPRNG token generation, HMAC-SHA256 token hashing (`phone:otp`), timing-safe verification, 5-minute expiry window, 60-second resend cooldown, sliding-window rate limit (max 3 dispatches per 10 minutes), and 3-attempt brute-force lockout.
- Native OAuth 2.0 PKCE Flow: RFC 7636 PKCE implementation (`code_verifier` with SHA-256 `code_challenge`, state-based CSRF protection with secure HttpOnly cookies), supporting Google Workspace SSO for staff and Google Sign-in for parents.
- DPDP Act (2023) Compliance Gate: Mandatory verifiable parental consent capture before granting portal access to minor students' academic, attendance, or financial records. Audit trails record parent identifier, timestamp, IP address, and consented child IDs.
- 4-Role Role-Based Access Control (RBAC):
- `PARENT`: Scoped strictly to consented linked children; zero access to institute-wide ledgers or other students.
- `ACCOUNTS`: Scoped to assigned Jaipur branches; authorized for ledger operations, payment allocations, receipt issuance, and withdrawal review.
- `COUNSELLOR`: Scoped to student risk assessment and academic intervention; strictly excluded from financial ledgers, payment details, and collection actions.

- `DIRECTOR`: Global multi-centre oversight; sole authority for maker-checker approval on refunds and fee waivers; audit log inspection.

2.3 Client/Server Trust Model

- Zero Client Trust: Browser redirects and frontend checkout completions land exclusively on a transitional Confirming state (`P1e`). No payment is acknowledged, no ledger entry posted, no receipt issued, and no reminder skipped based on frontend callbacks.
- Authoritative Gateway Verification: Ledger state transitions occur only upon receipt and cryptographic verification of server-side HMAC-SHA256 webhook payloads from the payment gateway or authoritative server-to-server reconciliation.

3. Data Model & Financial Invariants

3.1 Integer Paise (`bigint`) Representation

All monetary values across the database schema, domain models, APIs, and user interfaces are stored and processed strictly as 64-bit integer paise (`bigint`). Floating-point numbers are prohibited in all financial paths. For example, ` ¹30,000.00` is represented as `3000000n` paise. Rounding is performed using banker's rounding / round-half-up on integer divisions.

3.2 Append-Only Signed Ledger (`ledger_entries`)

Balances and overdue durations are derived strictly from signed immutable ledger entries:

- `CHARGE`: Debits representing scheduled tuition instalments.
- `DISCOUNT`: Credits representing approved scholarship or sibling concessions.
- `ALLOCATION`: Credits representing verified gateway payments matched to specific instalments.
- `REVERSAL`: Offsetting entries correcting previously posted allocations.
- `WAIVER`: Credits waiving remaining unpaid instalments upon approved student withdrawal.
- `REFUND_ADJUSTMENT`: Entries recording statutory refunds debited against the student ledger.

Historical ledger rows are never updated or deleted. Corrections are recorded exclusively through offsetting journal entries, preserving complete forensic auditability.

3.3 Database-Level Concurrency & Locking

To prevent race conditions during concurrent payment allocations, webhook replays, and refund executions across multiple server instances:

- PostgreSQL `SELECT ... FOR UPDATE` row-level locks are acquired on Payment and Withdrawal records within ACID database transactions before evaluating available funds or allocating balances.
- In-memory `financialLock` mutexes provide fast-path serialization within single-instance runtimes.

3.4 Balanced Reconciliation Invariant

The institute's financial ledger satisfies the dual-entry accounting equation at all times:

- Total Captured Payments - Total Approved Refunds = Total Allocated Balances + Total Unallocated Balances.
- Verified across all four Jaipur centres: ¹4,92,45,000 (Captured) - ¹3,30,000 (Refunded) = ¹4,89,15,000 (Net Funds) = ¹4,82,90,000 (Allocated) + ¹6,25,000 (Unallocated).

4. Comprehensive Operational Workflows

4.1 Parent Portal: Authentication, Dues & Checkout (Milestones 1–3)

- Login & Consent (`P0a`–`P0c`): Parent authenticates via MSG91 OTP widget or SMS fallback, passes the DPDP consent verification gate, and enters the mobile portal.
- 5-Second Dues Overview (`P1`): Features child switcher, English/Hindi language toggle, Hero Amount display with 52px tabular numerals, overdue status chip, signature `<Rail />` progress component, and primary Pay action above the 640px fold.
- Order Creation & Intent Binding (`P1c`): Generates a server-signed Razorpay Order and binds it to a durable `PaymentIntent` record with strict amount bounds and a 30-minute validity window.
- HMAC Webhook Processing (`P1e`): Webhook verifier validates the `X-Razorpay-Signature` HMAC-SHA256 header. Upon valid `payment.captured` event, the system idempotently commits the payment, posts ledger

allocations, generates a GST-compliant receipt (`P3`), and automatically marks pending instalment reminders as `SKIPPED`.

4.2 Statutory MoE 2024 Refund Engine (Milestone 4A)

- Ministry of Education Guidelines (Jan 2024): Strictly enforces pro-rata refund calculations for coaching institute withdrawals.
- Mathematical Formulation:
- `T`: Total course duration in calendar days (inclusive of start and end dates).
- `c`: Consumed days up to withdrawal date (inclusive).
- `N`: Net tuition fee after authorized institutional discounts.
- `earned_tuition = roundHalfUp(N * c / T)`.
- Non-refundable registration/admission fee: Set to ¹0 per statutory guideline.
- `maximum_refundable_amount = max(0, total_collected_fees - earned_tuition)`.
- Maker-Checker Authorization: Enforces strict administrative separation: `approverId !== requesterId`. Accounts Executives submit withdrawal requests; only the Founder-Director can approve or reject.
- Informational Shortfall Safeguard: In part-paid scenarios where tuition earned exceeds collected fees (`earned > paid`), the difference is displayed in neutral slate (`#505A66`) as an informational review metric. It is never treated or pursued as collectible debt.
- Withdrawal Ledger Rebase: Upon director approval, unpaid future instalments are automatically credited with `WAIVER` ledger entries, reducing the student's active balance to zero.

4.3 Real Razorpay Normal Refund API Integration (Milestone 4B)

- Direct Gateway Execution: Executes real HTTP `POST /v1/payments/:id/refund` requests against original captured payment transaction IDs in Razorpay Test Mode.
- Multi-Payment Refund Attribution: Intelligently allocates refund obligations across multiple historical payment transactions, respecting per-payment capturable ceilings and reversing allocations in LIFO/FIFO order.
- Razorpay Idempotency Contract: Implements durable idempotency keys (`refitem_<refundItemId>`) matching official gateway specifications. Ambiguous network timeouts maintain `PROCESSING` state without premature failure or duplicate execution.
- Refund Webhook Reconciliation: Handles `refund.speed_processed`, `refund.processed`, and `refund.failed` events to transition internal refund items and update parent ledger records.
- Aggregate Status State Machine: Evaluates composite parent refund status:
- All items `SUCCEEDED` -> `COMPLETED`.
- At least one item `SUCCEEDED` with others `PENDING`/`PROCESSING`/`FAILED` -> `PARTIALLY_REFUNDED`.
- Zero successful items, at least one `PROCESSING` -> `PROCESSING`.
- All items `FAILED` -> `FAILED`.

4.4 Unmatched Payment Investigation & Allocation (Milestone 5A)

- Suspension Queue Workspace (`/staff/payments`): Accounts workspace displaying captured payments that lack an automatic order match or retain unallocated balances. Displays student name, centre, captured amount, provider reference, and matching status.
- Heuristic Candidate Matching Engine: Evaluates unallocated payments against student records using a composite scoring engine:
- Exact reference / Order ID match: 100 points.
- Name similarity (Levenshtein / normalized token overlap): up to 40 points.
- Amount match to outstanding instalment: up to 40 points.
- Sibling / phone number linkage: up to 20 points.
- Authoritative Capture Provenance: Manual allocation requires verified payment provenance (`payment.captured` event with matching provider ID, amount, and currency). Unverified or arbitrary claims are rejected.
- Audit Logging: Every manual allocation logs the acting executive's ID, timestamp, target instalment, allocated amount, and mandatory justification note.

4.5 Staff Reminder Operations & Persistent Dispatch (Milestone 5B)

- Reminder Operations Workspace (`/staff/reminders`): Full visibility into queued, sent, failed, and skipped fee

reminders across all Jaipur branches.

- Evaluate-at-Send Architecture: The dispatcher queries live ledger balances at the exact moment of transmission. Stale queued figures are discarded; partially paid instalments quote the updated remaining balance.
- Stop-on-Settlement & Withdrawal Safety: Reminders are automatically suppressed if the target instalment has been settled, waived, or if the student has withdrawn.
- Persistent Atomic Claiming: Database-backed atomic transition from `QUEUED` to `PROCESSING` guarantees that only one server instance can claim and dispatch a reminder. Ambiguous network timeouts are safely flagged as `PROCESSING` rather than retried blindly.
- Quiet Hours Enforcement: Automated dispatches are paused between 21:00 and 08:00 IST to prevent parental harassment.
- Bilingual Message Templates: Dynamic template rendering supporting both English and Hindi communication channels.

4.6 Reporting, Reconciliation & Bulk Imports (AT-11 to AT-13)

- AT-11 Dashboard Ageing Buckets: Visual fee collection and ageing bars categorizing overdue dues into `1–30 days` (#8CBDB7), `31–60 days` (#F0C879), and `>60 days` (#E5534B), with interactive drill-down to underlying ledger-backed student records.
- AT-12 Academic Bulk Imports: 4-step import lifecycle (Upload, Validate, Review, Commit) with row-level error tagging (`INVALID`, `CHANGED`, `VALID_NEW`), formula injection sanitization, and downloadable error CSV reports.
- AT-13 Month-End Reconciliation & Tally Export: Formal month-close locking mechanism, balanced voucher generation, and deduplicated RFC-4180 CSV export formatted for seamless Tally ERP integration.

5. AI / Dropout-Risk Engine & Safety Controls

5.1 Explainable Heuristic Model

FeeFlow rejects unexplainable black-box models in favor of a deterministic, transparent composite risk score:

- Overdue fee duration (Weight `0.40`): Linear scaling from 0 to 60+ days overdue.
- Attendance trend (Weight `0.30`): 4-week moving average compared against the 8-week baseline.
- Academic performance trend (Weight `0.30`): Recent 3 test score average compared against the prior 3 tests.

5.2 Insufficient Data Guardrail

If a student has fewer than 2 active academic or attendance signals, the engine explicitly outputs `INSUFFICIENT_DATA` rather than generating a falsely reassuring `LOW` risk classification.

5.3 Safety Architecture & Non-Interference

- Zero Financial/Parent Code Paths: The risk evaluation engine has zero access to payment systems, ledger mutation routines, or automated messaging dispatchers.
- Counsellor-Only Action Queue: High-risk classifications (`>= 60`) create an internal `COUNSELLOR_CALL` review task on `/staff/counsellor`.
- 30-Day False Positive Suppression: Counsellors can dismiss false alerts with a mandatory recorded reason, updating accuracy metrics and suppressing repeated alerts for 30 days.

6. UI & Design System Philosophy

6.1 Parent Portal UX

- Designed specifically for 360px mobile viewports to accommodate parents using entry-level smartphones.
- High-contrast Hero Amount with tabular numerals for instantaneous comprehension.
- The signature `<Rail />` component provides clear visual progress across 3-instalment (NEET) and 4-instalment (JEE) structures.
- One-tap bilingual toggle between English and Hindi (`EN | “™?•& ’à

6.2 Staff Administrative UX

- High-density desktop layouts (`1280px`+) optimized for administrative throughput.
- Instant search and multi-facet filtering across centres, batches, and payment statuses.
- Slide-out detail drawers providing full audit histories and transaction drill-downs without page context switching.

- Standardized color system: Deep Teal (`#0B5D5A`) for primary actions, Warm Paper (`#FAF8F4`) for backgrounds, Ink (`#17191A`) for typography, and Neutral Slate (`#505A66`) for informational shortfalls.

7. Labelled Assumptions & Operational Boundaries

- [A-01] Demo Business Clock: Fixed at `2026-11-02` (Asia/Kolkata) to deterministically reproduce the brief sample cases (`STU-0587` Ishita Sharma 23 days overdue, `STU-0219` Mohit Gurjar due in 13 days).
- [A-02] Course Fee Schedules: Standard annual fee of `₹1,20,000` structured as 4 × `₹30,000` quarterly instalments for JEE programs and 3 × `₹40,000` instalments for NEET programs.
- [A-03] Minimum Payment Threshold: Minimum allowable online partial payment is set to `₹1,000` (or the exact remaining instalment balance if lower).
- [A-04] MoE Statutory Calendar: Pro-rata calculations use inclusive calendar day counting (`T` and `c`) and ₹0 registration deductions; 10-day refund turnaround is tracked as an operational SLA.
- [A-05] Tax Compliance Metadata: Automated GST receipts feature the institute's Jaipur GSTIN (`08AABCC1234D1Z5`), SAC Code `999293`, and itemized 18% GST (9% CGST + 9% SGST) breakdown.

8. What Was Cut & What Comes Next

8.1 Scoped Out for Take-Home Submission

- Live production WhatsApp Business API integration (mock notification provider implemented).
- Direct bank NEFT/RTGS automated reconciliation via Open Banking APIs (manual allocation workspace implemented).
- Multi-tenant institute partitioning (single institute with multi-centre hierarchy implemented).
- Student Learning Management System (LMS) and admissions CRM (outside problem boundary).

8.2 Production Roadmap

- Connection of live WhatsApp Business API and production Razorpay merchant webhooks.
- Formal legal and Chartered Accountant sign-off on DPDP privacy disclosures and GST credit note series numbering.
- Machine learning model calibration on multi-year Jaipur historical cohort retention datasets.

9. Comprehensive Automated Test Verification

FeeFlow is validated by an exhaustive automated test suite achieving 100% green execution across 28 test files and 295 individual tests.

9.1 Test Suite Breakdown

1. `razorpay-refund.test.ts` (43 tests): Real Razorpay Normal Refund API integration, request signing, idempotency contracts, multi-payment refund attribution, network timeout handling, and composite parent refund state aggregation.
2. `payment-settlement-webhook.test.ts` (25 tests): Webhook signature verification, idempotent event processing, ledger allocation, GST receipt creation, and reminder suppression.
3. `msg91-widget.test.ts` (21 tests): MSG91 authentication widget integration, token exchange, and phone number verification.
4. `payment-manual-allocation.test.ts` (17 tests): Unmatched payment workspace, candidate scoring, manual allocation with reason, capture provenance verification, and cross-instance concurrency locking.
5. `withdrawal-refund-workflow.test.ts` (16 tests): Full end-to-end withdrawal lifecycle, maker-checker authorization, ledger rebase, waiver posting, and refund execution.
6. `reminder-operations.test.ts` (16 tests): Staff reminder workspace, evaluate-at-send balance revalidation, persistent atomic dispatch claiming, quiet hours, and provider isolation.
7. `payment-intent.test.ts` (15 tests): Razorpay order generation, intent binding, amount validation, 30-minute expiry window, and zero-client-trust invariants.
8. `payment-sandbox-matrix.test.ts` (14 tests): Full 13-scenario payment sandbox test matrix covering network drops, replays, currency mismatches, and partial payments.
9. `otp-service.test.ts` (14 tests): E.164 normalization, CSPRNG generation, HMAC-SHA256 hashing, timing-safe verification, cooldowns, rate limits, and brute-force lockout.

10. `auth-api.test.ts` (11 tests): Parent and staff authentication endpoints, session cookie management, and role-based redirect routing.
11. `withdrawal-api.test.ts` (10 tests): Withdrawal request submission, director approval/rejection endpoints, and permission enforcement.
12. `razorpay.test.ts` (10 tests): Razorpay payment gateway integration, order creation, and signature validation.
13. `refunds.test.ts` (9 tests): Statutory MoE pro-rata formula, golden fixtures F1–F4, UI fixtures U1–U2, and informational shortfall safeguards.
14. `payment-hardening.test.ts` (8 tests): Financial boundary testing, negative amount rejection, and overflow protection.
15. `csv-sanitizer.test.ts` (7 tests): RFC-4180 compliance and spreadsheet formula injection sanitization (`=`, `+`, `-`, `@`, `\t`, `\r`).
16. `ledger.test.ts` (6 tests): Append-only signed ledger derivation, balance integrity, and status transitions.
17. `money.test.ts` (6 tests): BigInt integer paise conversion, banker's rounding, and currency formatting boundaries.
18. `rbac.test.ts` (6 tests): Role permissions, centre scoping, DPDP consent gating, counsellor ledger exclusion, and maker-checker validation.
19. `oauth-provider.test.ts` (5 tests): PKCE code verifier and challenge generation, CSRF state protection, and profile normalization.
20. `oauth-api.test.ts` (5 tests): Native OAuth PKCE authorization routes, state cookie verification, and session creation.
21. `reminders.test.ts` (5 tests): Domain reminder evaluation, stop-on-settlement, and quiet hours filtering.
22. `components.test.tsx` (5 tests): Component rendering for `<Rail />`, StatusChips, ProvenanceChips, and DemoBar.
23. `risk.test.ts` (4 tests): Explainable 0.40/0.30/0.30 heuristic, sparse data safety guardrails, and 30-day false positive suppression.
24. `payment-webhook.test.ts` (4 tests): Gateway webhook signature validation and tampering detection.
25. `sms-provider.test.ts` (4 tests): SMS provider interface, message formatting, and delivery logging.
26. `parent-nav.test.tsx` (4 tests): Parent mobile navigation accessibility, route indicators, and bilingual labeling.
27. `matching.test.ts` (3 tests): Unmatched payment candidate scoring engine (ref/name/amount heuristics).
28. `parent-registration.test.ts` (2 tests): End-to-end parent self-registration, student linkage, and OTP verification.

9.2 Financial Invariants Certified

1. Money Precision: 100% integer paise throughout database and application layers; zero floating-point arithmetic.
2. Ledger Primacy: Balances are derived exclusively from immutable signed ledger entries.
3. Zero Client Trust: Browser redirects land on Pending only; only HMAC-verified webhooks mutate the ledger.
4. Pro-Rata Refund Safeguard: Statutory MoE 2024 formula enforced; informational shortfalls never pursued as debt.
5. Maker-Checker Separation: Server-enforced `approverId !== requesterId` on all refund and waiver decisions.
6. CSV Injection Hardening: All exported spreadsheets sanitized against spreadsheet formula execution.
7. Dual-Entry Reconciliation: Net Captured Funds strictly equals Allocated Balances plus Unallocated Suspense.